Assignment 1: Leaf Image Classification with an MLP (LeavesDataset)

📌 Important Information

- **Deadline**: April 7, 2026, by 23:59 (`Beijing Time`)
- **Submission Format**: Submit your answer via the SUSTech Blackboard system. You should submit:
  - **Code**: `(Assignment1_{student ID}.zip)`
  - **A Short report**: `(Assignment1_{student ID}.pdf)`

ℹ️ Introduction

In this assignment, you will implement an image classification task using PyTorch to train a Multi-Layer Perceptron (MLP) on a Leaf Dataset. The task is to predict categories of leaf images. This dataset contains 176 categories, with 18,353 images. This assignment will help you understand deep learning fundamentals, hyperparameter tuning, and the implementation of a custom dataset pipeline using PyTorch.

❓ Environment Setup

1. **Training**:
   - Use the provided `LeavesDataset` folder, which is split into training (70%), validation (10%), and test (20%) sets based on the train.csv file.
   - Implement an MLP model to classify images, trained using PyTorch.
   - **Do not tune hyperparameters on the test set.** Use training curves and validation set for model selection.
2. **Data Splitting**:

- The dataset is divided using a 7:1:2 ratio:
  - **Training Set**: 70% of the data (12,847 images, approximately).
  - **Validation Set**: 10% of the data (1,835 images, approximately).
  - **Test Set**: 20% of the data (3,671 images, approximately).
- The split is implemented in the LeavesDataset class, ensuring deterministic separation of data based on the provided ratios.

3. **Code Details**:

- The code includes a custom `LeavesDataset` class (`dataset.py`) for loading and preprocessing leaf images, along with data loaders for training, validation, and testing.
- Image preprocessing includes resizing to 224x224, normalization (using ImageNet means and stds), and data augmentation (random horizontal flips for training).
- Noted that the code is just a guideline, you can replace it with a better implementation of you.

⭐ Assignment Instructions

1. Implement LeavesDataset Loading

- **Objective**: Load the provided `LeavesDataset` from disk and build PyTorch dataloaders for training and testing.
- **Requirements**:
  - The network must be a pure MLP (no convolutional layers allowed).
  - Use the provided `LeavesDataset` class and data loaders for training and evaluation.
  - Evaluate performance on the validation set during hyperparameter tuning, and use the test set only for final evaluation.
- **Suggested directions**:
  - You are encouraged to experiment with different DataLoader settings, such as `batch_size` , `shuffle` , `num_workers` , and `pin_memory` , to improve training efficiency and model performance.

2. Hyperparameter Analysis

- **Objective**: Investigate the impact of various hyperparameters on model performance.
- **Requirements**:
  - Train the model with different hyperparameter configurations.
  - Analyze the impact of key hyperparameters, including:
    - **Number of Hidden Layers**: Modify the MLP architecture to vary the number of hidden layers (e.g., 1 to 6).
    - **Hidden Dimension**: Compare different widths (e.g., 128, 256, 512, 1024).
    - **Learning Rate**: Test different values (e.g., 1e-1, 1e-2, 1e-3, 1e-4, 1e-5).
    - **Weight Decay**: Experiment with values (e.g., 0, 1e-5, 1e-4, 1e-3).
    - **Optimizer**: Compare optimizers like SGD vs Adam.
    - **Loss Function**: Use a standard classification loss (e.g., CrossEntropyLoss).
    - **Epochs**: Experiment with different numbers of training epochs (e.g., 10, 20, 50, 100).
    - **Batch Normalization**: Add BN layers to stabilize training.
    - Dropout: Introduce dropout layers (e.g., p=0.2, 0.5) to prevent overfitting.

    - Record training and validation accuracy/loss curves (saved via `plot_metrics` in `utils.py`).
    - Discuss how different hyperparameter choices affect performance.
- **Suggested directions**:
    - You are encouraged to experiment with different learning rate and weight decay schedules to improve training performance.
    - You are encouraged to compare different optimizers and tune their internal hyperparameters, such as momentum in SGD and betas in Adam.
    - You are also encouraged to explore different loss functions, when appropriate, to see whether they improve performance.
    - For each modification, briefly explain why it may help and support your discussion with empirical results.

3. Evaluation and Error Analysis

- **Objective**: Evaluate generalization performance, justify appropriate evaluation metrics, and analyze common failure modes.
- **Requirements**:
    - Choose the evaluation metric(s) you believe are most appropriate for this task, and explain why you selected these metrics.
    - Through experiments, investigate whether your modeling choices or training strategies improve performance under the chosen metric(s).
- **Suggested directions**:
    - If the dataset is approximately balanced and all classes are equally important, accuracy may be a reasonable choice.
    - If the dataset is imbalanced, metrics such as Macro-F1 may be more informative because they give more balanced attention to all classes.
    - If some classes are especially important or particularly difficult to distinguish, you may also consider class-wise precision, recall, or F1.

📝 Report Guidelines

- **Content**:
    - Present hyperparameter analysis results (tables/plots of accuracy/loss vs. hyperparameters).
    - Include learning curves for representative runs and discuss overfitting/underfitting.
    - Carefully design tables to summarize your experimental settings and results. Think critically about what information is essential, what is unnecessary noise, and how to present the results so that the table can support systematic comparison and guide the next round of improvements in your experiments.

- **Format**: PDF, no more than 3 pages, including figures and tables.

💡 Tips

- Use the `--seed` argument to ensure reproducibility.
- Keep experiment naming consistent so your results are easy to trace back to configurations.